CE 222 – *Computer Organization and Assembly Language (COAL)*
Lecture 15

Dr. Asad Mahmood

Feb. 18, 2026

Lecture Outline

- Announcements (Assignment and Quiz 2 this week)
- Outline of Previous Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.8 Supporting Procedures in Computer Hardware
    - 2.9 onwards – Mostly Reading

- Outline to Today's Lecture:
  - **Chapter 3 – Arithmetic for Computers**
    - 3.1 Introduction
    - 3.2 Addition and Subtraction
    - 3.3 Multiplication

Chapter 2

Instructions: Language of the Computer

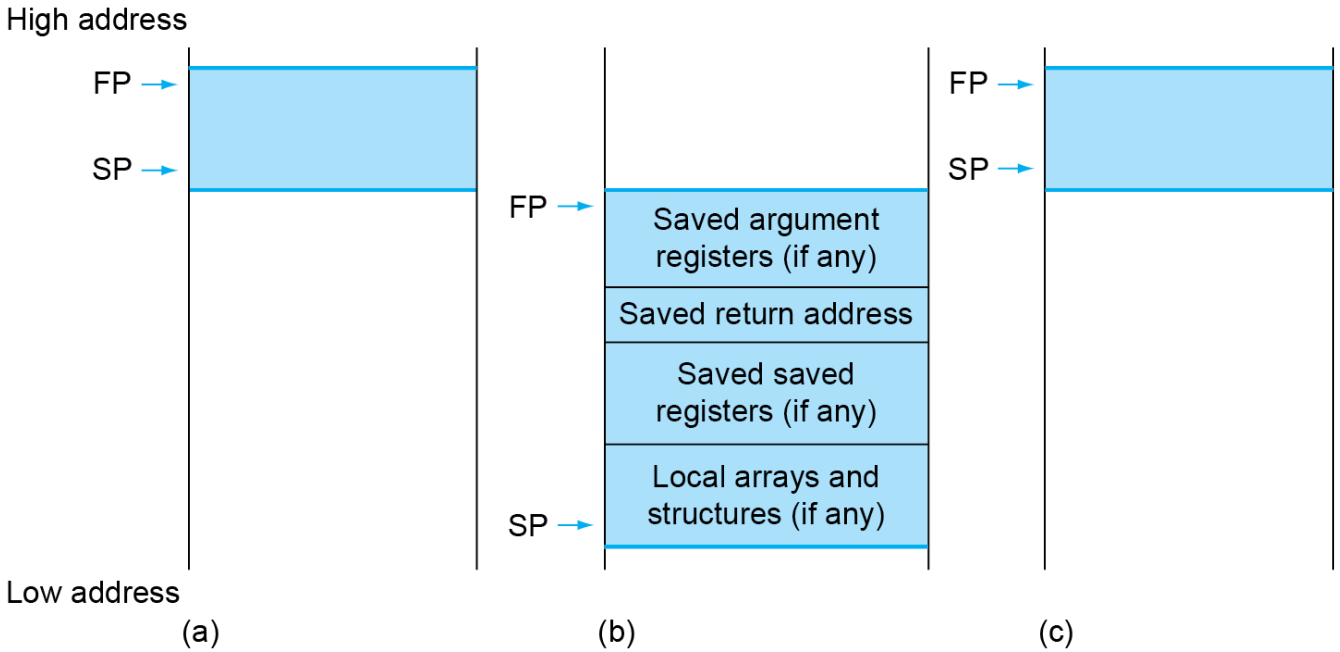
Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Register and Memory Preservation

Preserved	Not preserved
Saved registers: x8-x9, x18-x27	Temporary registers: x5-x7, x28-x31
Stack pointer register: x2(sp)	Argument/result registers: x10-x17
Frame pointer: x8(fp)	
Return address: x1(ra)	
Stack above the stack pointer	Stack below the stack pointer

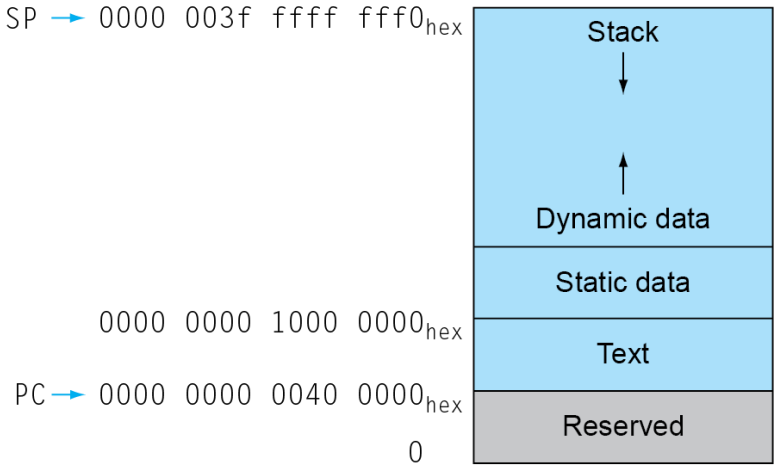
Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - x3 (global pointer) initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



Tail Recursions

- Efficient Implementation possible

- Example (C Code)

```
int sum (int n, int acc)
{
  if (n > 0)
       return sum(n – 1, acc + n);
  else
       return acc;
}
```

  - x10 =n, x11 = acc, x12 = result/return value

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Byte/Halfword/Word Operations

- RISC-V byte/halfword/word load/store
  - Load byte/halfword/word: Sign extend to 32 bits in rd
    - `lb rd, offset(rs1)`
    - `lh rd, offset(rs1)`
    - `lw rd, offset(rs1)`
  - Load byte/halfword/word unsigned: Zero extend to 32 bits in rd
    - `lbu rd, offset(rs1)`
    - `lhu rd, offset(rs1)`
    - `lwu rd, offset(rs1)`
  - Store byte/halfword/word: Store rightmost 8/16/32 bits
    - `sb rs2, offset(rs1)`
    - `sh rs2, offset(rs1)`
    - `sw rs2, offset(rs1)`

String Copy Example

- C code:
  - Null-terminated string

```
void strcpy (char x[], char y[])
{    size_t i;
     i = 0;
     while ((x[i]=y[i])!='\0')
     i += 1;
}
```

- Assume base addresses of x[] and y[] in x10 and 11, respectively. ‘i’ in x19

String Copy Example

- RISC-V code:

```
strcpy:
        addi sp,sp,-4          // adjust stack for 1 word
        sw   x19,0(sp)         // push x19 to stack
        add  x19,x0,x0         // i=0

L1:  add  x5,x19,x10           // x5 = addr of y[i]
        lbu  x6,0(x5)          // x6 = y[i]
        add  x7,x19,x10        // x7 = addr of x[i]
        sb   x6,0(x7)          // x[i] = y[i]
        beq  x6,x0,L2          // if y[i] == 0 then exit
        addi x19,x19,1         // i = i + 1
        jal  x0,L1             // next iteration of loop

L2:  lw   x19,0(sp)            // restore saved x19
        addi sp,sp,4           // pop 1 word from stack
        jalr x0,0(x1)          // and return
```

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

32-bit Constants

- Most constants are small
  - 12-bit immediate is sufficient
- For the occasional 32-bit constant

  `lui rd, constant` (U-type!)
  - To copy a 32 bit constant, first its left-most 20 bits are copied to bits [31:12] of rd using `lui`
  - Then the rest 12 bits are added
  - Example

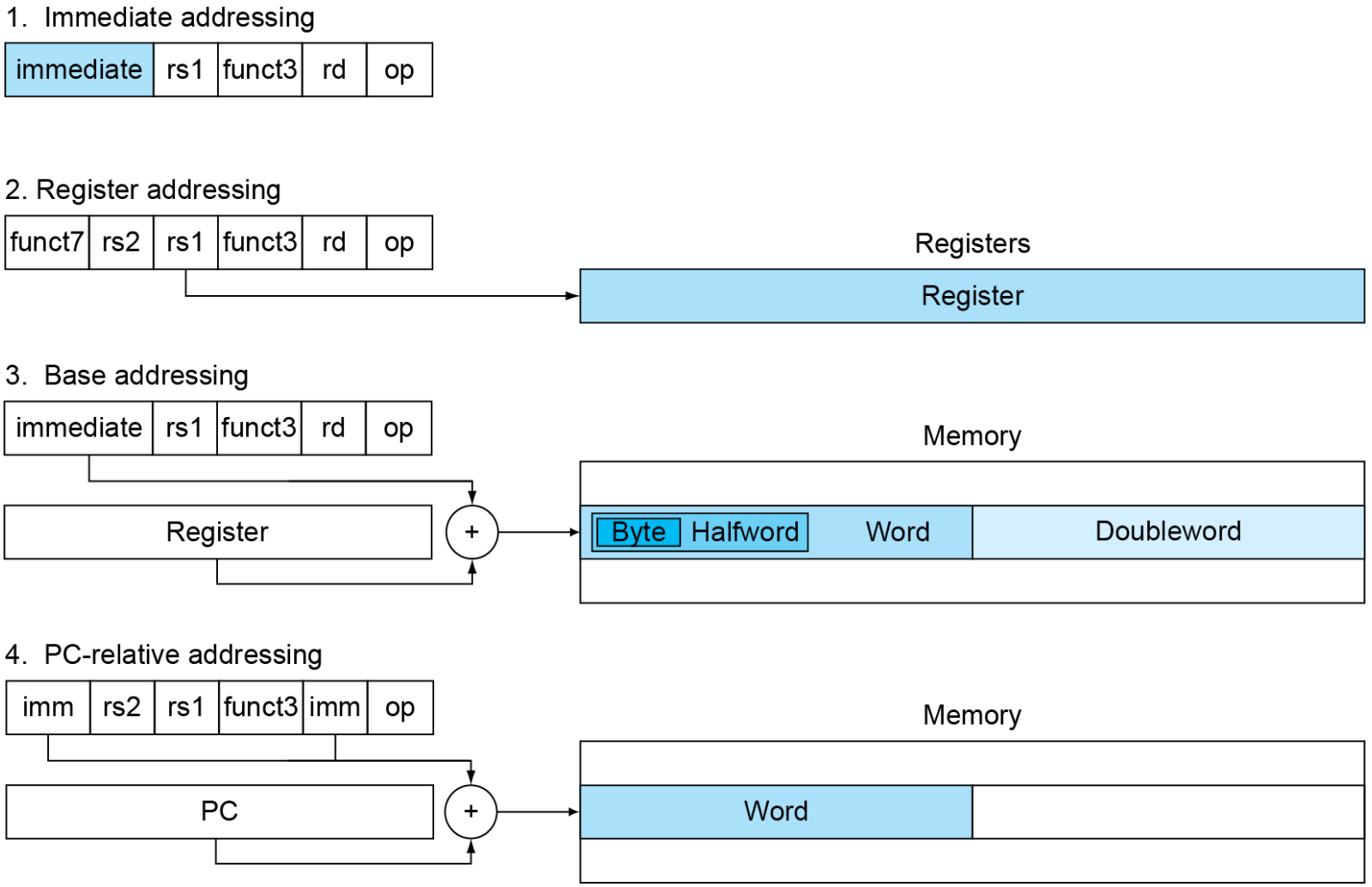

```
lui x19, 976  // 0x003D0
```

0000 0000 0011 1101 0000	0000 0000 0000

```
addi x19,x19,1280  // 0x500
```

0000 0000 0011 1101 0000	0101 0000 0000

RISC-V Addressing Summary



RISC-V Encoding Summary

Name (Field Size)	Field 7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

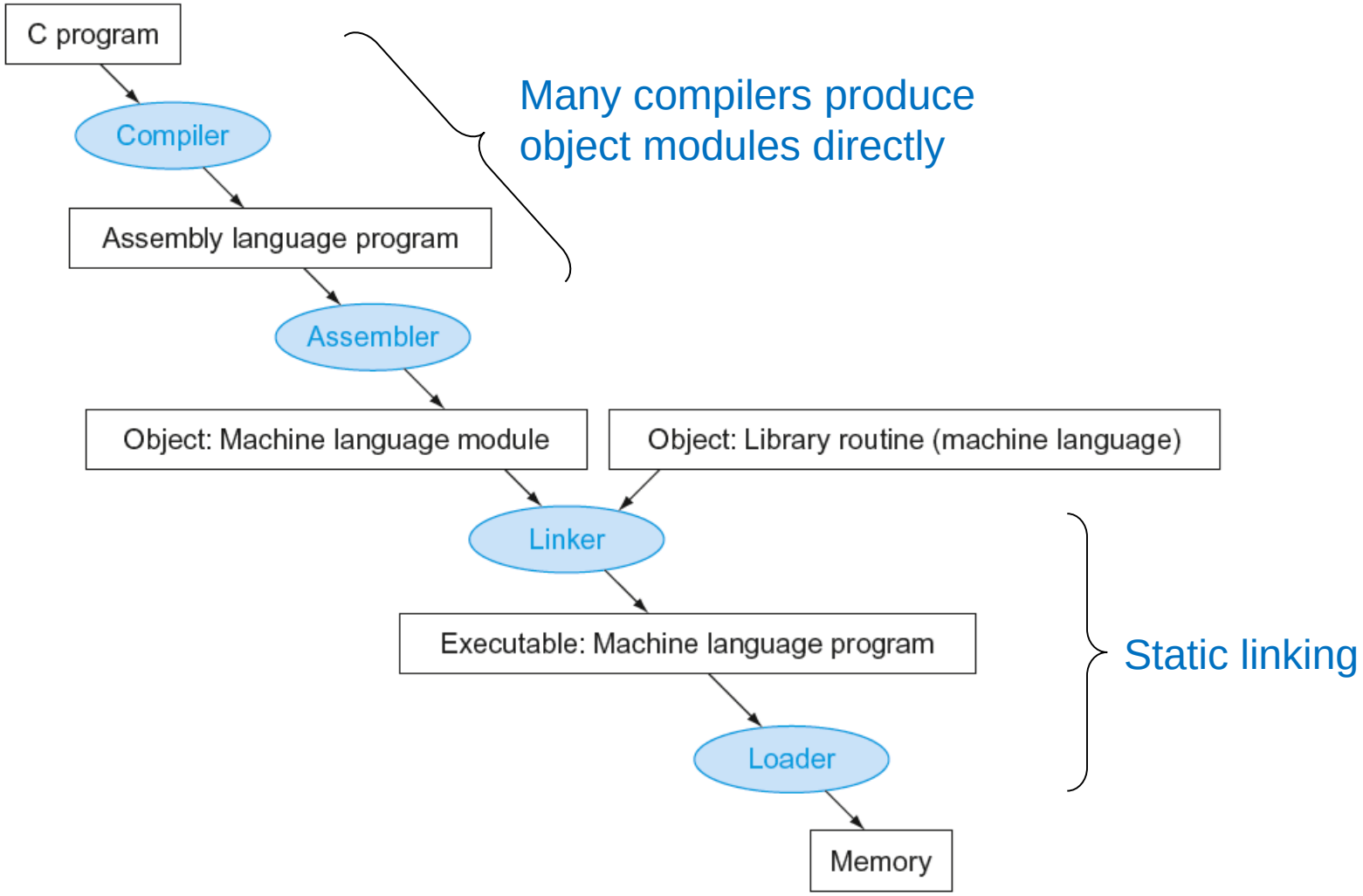
Synchronization

- Two processors sharing an area of memory
  - P1 writes, then P2 reads
  - Data race if P1 and P2 don’t synchronize
- Hardware support required
  - Atomic read/write memory operation
  - No other access to the location allowed between the read and write
- Could be a single instruction
  - E.g., atomic swap of register ↔ memory
  - Or an atomic pair of instructions

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

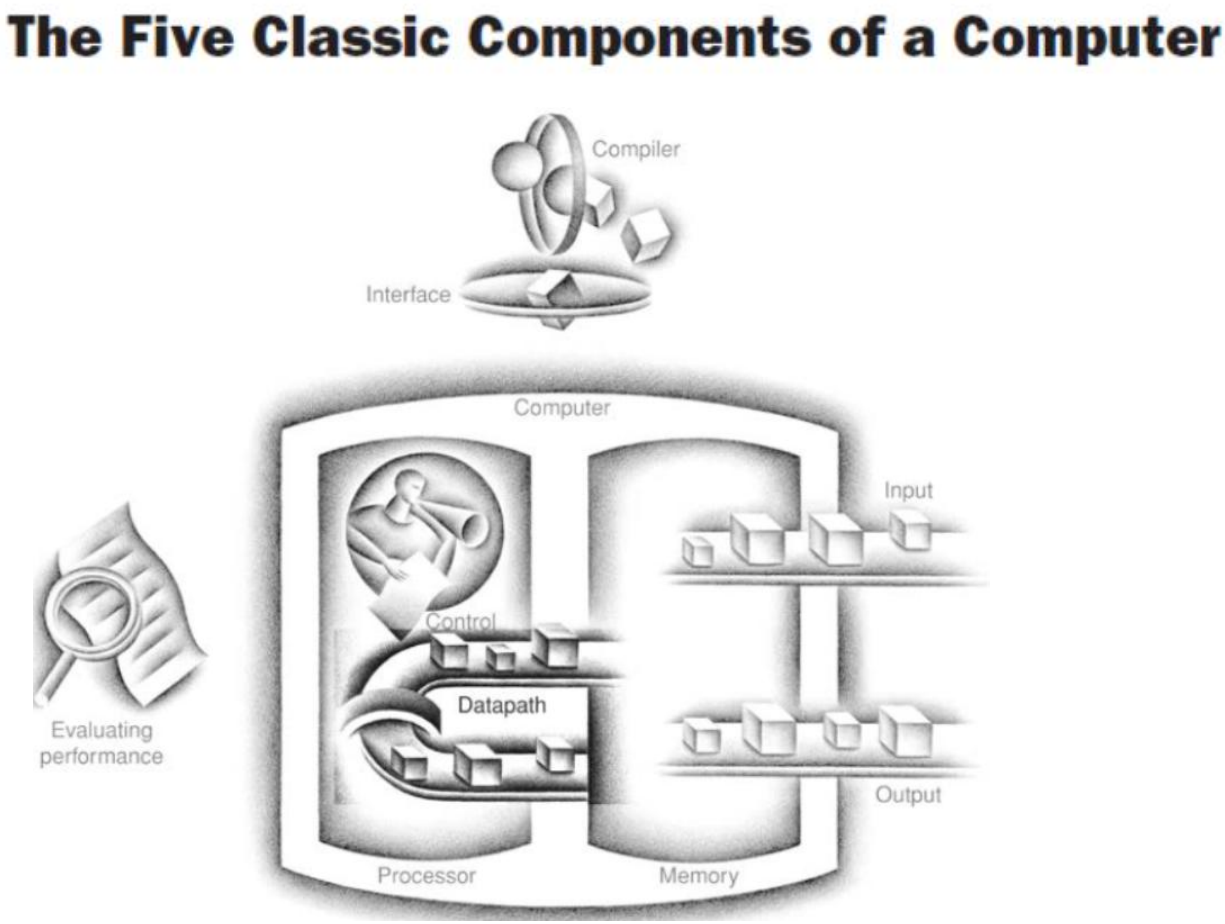
Translation and Startup



Chapter 3

Arithmetic for Computers

Chapter Theme



Introduction

Numerical precision is the very soul of science.

Sir D'arcy Wentworth Thompson,

On Growth and Form, 1917

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Introduction

- Computer operates on integers using their binary representation
- Arithmetic operations are very important for a processor
- Common arithmetic operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
- Dealing with real numbers
- Implications of operations implementations on ISA

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Integer Addition

- As studied in logic design
- Example: 7 + 6
- Overflow
  - When adding +ve and –ve operands
  - When adding two +ve or two –ive operands
  - How to detect?

Integer Subtraction

- Add negation of second operand
- Example: 7 – 6

- Overflow
  - Subtracting two +ve or two –ve operands
  - Subtracting +ve from –ve operand
  - Subtracting –ve from +ve operand

Arithmetic Operations

- With RISC-V 32-bit registers, we will need an extra bit to detect overflows
- Overflow in unsigned nos.
- Some languages like C may not give correct answer when overflow happens -> Solution?
- Implementations of these arithmetic operations greatly impact processor performance

Arithmetic for Multimedia

- Graphics and media processing operates on vectors of 8-bit and 16-bit data
  - Use 64-bit adder, with partitioned carry chain
    - Operate on 8×8-bit, 4×16-bit, or 2×32-bit vectors
  - SIMD (single-instruction, multiple-data)
- Saturating operations
  - On overflow, result is largest representable value
    - c.f. 2s-complement modulo arithmetic
  - E.g., clipping in audio, saturation in video

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading